Chapter 3 worked solutions

DS03-E01 • Circular wraparound

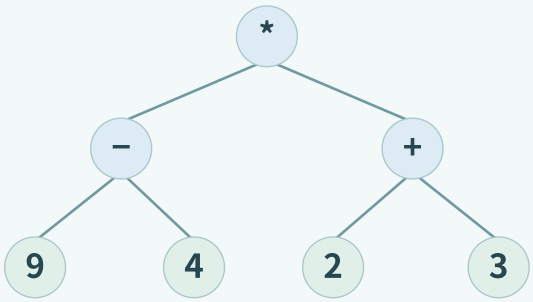
Initially $(head, size) = (0, 0)$. A dash means that no live element occupies the slot. Any leftover bits in a removed slot are outside the abstract sequence. Enqueue uses slot $(head + size) \bmod 4$; dequeue sets $head = (head + 1) \bmod 4$.

Completed operation	head	size	Slot 0	Slot 1	Slot 2	Slot 3	Logical sequence
Initial	0	0	–	–	–	–	Empty
Enqueue A	0	1	A	–	–	–	A
Enqueue B	0	2	A	B	–	–	A,B
Enqueue C	0	3	A	B	C	–	A,B,C
Dequeue A	1	2	–	B	C	–	B,C
Dequeue B	2	1	–	–	C	–	C
Enqueue D	2	2	–	–	C	D	C,D
Enqueue E	2	3	E	–	C	D	C,D,E
Enqueue F	2	4	E	F	C	D	C,D,E,F
Dequeue C	3	3	E	F	–	D	D,E,F
Dequeue D	0	2	E	F	–	–	E,F
Dequeue E	1	1	–	F	–	–	F

Full means size=4; empty means size=0. This representation does not use equality of head and a tail index to distinguish the two states.

DS03-E02 • Expression conversion

The root is *. Its left subtree is − with leaves 9 and 4. Its right subtree is + with leaves 2 and 3.



Prefix: root → left → right

* − 9 4 + 2 3

Postfix: left → right → root

9 4 − 2 3 + *

(9−4) × (2+3) = 5 × 5 = 25

Postfix evaluation: pop the right value before the left value

Prefix is `* - 9 4 + 2 3`. Postfix is `9 4 - 2 3 + *`. The postfix stack states are `[9]`, `[9,4]`, `[5]`, `[5,2]`, `[5,2,3]`, `[5,5]`, `[25]`. For `8 3 -`, the first pop 3 is the right operand and the second pop 8 is the left operand. The result is 5. Exchanging them gives −5 and changes the expression tree's meaning.

DS03-E03 • The first DFS path

Use directed edges `s→a,a→b,b→c,c→t,s→t`, with a preceding t in s's neighbor order. DFS visits s,a,b,c,t and first reaches t in four edges. Its parents are a←s,b←a,c←b,t←c. BFS discovers both a and t while processing s, so t has parent s and distance 1. Continuing the whole BFS can give visit order s,a,t,b,c, with b←a and c←b. This counterexample concerns DFS neighbor order, not its correctness for reachability.

The complete BFS assigns a←s, t←s, b←a, and c←b. Distances are d(s)=0, d(a)=d(t)=1, d(b)=2, and d(c)=3. Assign a parent only on first discovery. Encountering the already discovered t from c does not change its parent to c.

DS03-E04 • Syntax and safe evaluation

Reject `2 +` when + finds fewer than two operands. Reject `2 3` because two values remain at the end. Reject `8 0 /` when the right operand is zero. Also check integer-token parsing range, addition, subtraction, and multiplication overflow, and the unrepresentable result of the minimum signed integer divided by −1. If unary minus exists, token type must specify its arity. Stack size cannot determine the grammar.

DS03-E05 • End-of-input and grammar

token	ops (bottom→top)	output
9	∅	9
−	−	9
4	−	9 4
*	− *	9 4
2	− *	9 4 2
/	− /	9 4 2 *
1	− /	9 4 2 * 1
+	+	9 4 2 * 1 / −
3	+	9 4 2 * 1 / − 3
EOF	∅	9 4 2 * 1 / − 3 +

At +, first pop higher-precedence /, then equal-precedence left-associative −. Evaluate 4×2=8, 8/1=8, 9−8=1, 1+3=4.

Input	Rejection point	Reason
(9−4	EOF	Unmatched left parenthesis remains after − pops
9−4)	Right parenthesis	No matching left parenthesis
9−	EOF	A right operand is still required

Right-associative `2 3 2 ^ ^` computes $3^2=9$ then $2^9=512$. Left-associative `2 3 ^ 2 ^` computes $2^3=8$ then $8^2=64$.

DS03-E06 • Queue growth and the abstract value

Move C from old slot 2 to new 0, D from 3 to 1, E from 0 to 2, and F from 1 to 3. Commit head=0,size=4,capacity=8. Copying old slots and retaining head=2 makes logical index 2 access `(2+2) mod 8=4`, an unconstructed slot rather than E. Logical-order relocation costs Θ(n), allowing Θ(n) worst-case enqueue. Doubling only when full with constant-time moves gives a geometric total of Θ(n) over n enqueues from empty, hence Θ(1) amortized. Nonconstant value costs must be added. This differs from a fixed-capacity worst-case constant bound.

DS03-E07 • Mixed operations with two stacks

All sequences are bottom-to-top. Initially I=O=[] and Φ=0. Actual costs below count stack pushes/pops, excluding constant empty checks.

Operation	I	O	FIFO	c	ΔΦ	c+ΔΦ
enqueue 4	[4]	[]	[4]	1	2	3
enqueue 7	[4,7]	[]	[4,7]	1	2	3
dequeue→4	[]	[7]	[7]	5	−4	1
enqueue 9	[9]	[7]	[7,9]	1	2	3
dequeue→7	[9]	[]	[9]	1	0	1
dequeue→9	[]	[]	[]	3	−2	1

The first dequeue performs two pops and two pushes for transfer, then pops 4: cost 5. The next dequeue removes 7 from O without moving 9. Only the last dequeue transfers 9. Actual and amortized totals both equal 12 because initial and final potential are zero. For an arbitrary prefix, nonnegative final potential makes actual cost at most the amortized total.

DS03-E08 • Stack permutations and required capacity

Target	Operations or first obstruction	Verdict
2,1,4,3	push 1,push 2,pop 2,pop 1,push 3,push 4,pop 4,pop 3	Feasible; maximum depth 2
3,1,4,2	push 1,push 2,push 3,pop 3; top 2 now blocks requested 1	Impossible; pushing 4 only adds an obstruction

The first target requires capacity at least 2: to output 2 before 1, both must coexist. The displayed operations attain that lower bound, proving minimum capacity 2.

$C_0=1, C_1=1, C_2=2, C_3=5$, so $C_4=C_0C_3+C_1C_2+C_2C_1+C_3C_0=5+2+2+5=14$. The 24 permutations counted by 4! include outputs obstructed by LIFO, such as the second target.

DS03-E09

For `([])`, stack states are `(`, `([`, `(`, empty, so accept. For `([)]`, at index 2 the `)` sees top `[`, so reject. `(()` ends with `(` still on the stack, so reject at end of input. The empty string leaves an empty stack and is accepted. `([)]` has equal counts of each kind but crosses rather than nests, so counts alone are insufficient.

DS03-E10

For `racecar`, push `r,a,c`, skip the middle `e`, and compare/pop `c,a,r`; all match. For `abca`, push `a,b` and reject when `c` differs from top `b`. Each successful comparison consumes one second-half symbol, so there are at most $\lfloor n/2 \rfloor$ comparisons; early rejection uses fewer. The single code point `é` is a palindrome, but its UTF-8 bytes are `C3 A9`, which differ, so the byte sequence is not a palindrome.

DS03-E11

`aabb` is accepted. `aab` leaves an unmatched `a`; the second `b` in `abb` has no marker; `abab` contains an `a` after entering the `b` phase. The empty string is rejected because $n \geq 1$. If $n \geq 0$, only the empty-string result changes among these five inputs.

DS03-E12

Head/size holds 4 values, with empty `size=0` and full `size=4`. The reserved-slot representation holds 3, with empty `front=rear` and full `(rear+1) mod 4=front`. The tag representation holds 4: when the indices coincide, tag=false means empty and tag=true means full. Head points to the first element; in the other two conventions front is just before it. Index meaning and empty/full rules form one contract and cannot be mixed independently.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft stack and queue; MIT 6.006, Lecture Notes 9–10.
Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study